STEP 1 — Notebook Setup

LAB-11
ROLL NO:2403A52397
NAME : VARSHITHA

STEP 2 — Import Libraries

NumPy and Pandas are used for data handling. NLTK is used to load and preprocess movie review text. TfidfVectorizer converts text into numerical features. LogisticRegression is used for classification. Sklearn metrics evaluate model performance. Matplotlib and Seaborn are used for visualization.

```
import numpy as np
import pandas as pd

import nltk
from nltk.corpus import movie_reviews
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_m

import matplotlib.pyplot as plt
import seaborn as sns

nltk.download('movie_reviews')
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data]   Unzipping corpora/movie_reviews.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
import pandas as pd

data = pd.read_csv("IMDB Dataset.csv")
data.head()
```

| | review | sentiment |
|---|---|---|
| 0 | One of the other reviewers has mentioned that ... | positive |
| 1 | A wonderful little production. <br /><br />The... | positive |
| 2 | I thought this was a wonderful way to spend ti... | positive |
| 3 | Basically there's a family where a little boy ... | negative |
| 4 | Petter Mattei's "Love in the Time of Money" is... | positive |

Next steps: Generate code with data | New interactive sheet

STEP 3 — Load and Preprocess Data

The IMDB dataset contains movie reviews labeled as positive or negative. The dataset consists of text reviews and corresponding sentiment labels. Preprocessing includes converting text to lowercase, removing punctuation and stopwords. This helps reduce noise and improve model performance.

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))

def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [w for w in tokens if w.isalpha()]
    tokens = [w for w in tokens if w not in stop_words]
    return " ".join(tokens)

data['clean_text'] = data['review'].apply(preprocess)

data[['clean_text', 'sentiment']].head()
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

| | clean_text | sentiment |
|---|---|---|
| 0 | one reviewers mentioned watching oz episode ho... | positive |
| 1 | wonderful little production br br filming tech... | positive |
| 2 | thought wonderful way spend time hot summer we... | positive |
| 3 | basically family little boy jake thinks zombie... | negative |
| 4 | petter mattei love time money visually stunnin... | positive |

STEP 4 — Apply TF-IDF Vectorizer

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical feature vectors. It assigns higher weight to words that are important in a document but less frequent across all documents. This helps the model focus on meaningful sentiment-related words. Limiting max_features to 5000 reduces dimensionality and improves efficiency.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF Vectorizer
vectorizer = TfidfVectorizer(max_features=5000)

# Transform clean text into numerical features
X = vectorizer.fit_transform(data['clean_text'])

# Target variable
y = data['sentiment']

print("TF-IDF Matrix Shape:", X.shape)
print("Vocabulary Size:", len(vectorizer.vocabulary_))
```

```
TF-IDF Matrix Shape: (50000, 5000)
Vocabulary Size: 5000
```

STEP 5 — Train Logistic Regression Model

The dataset is split into training and testing sets (80/20). Logistic Regression is trained on TF-IDF features. It uses the sigmoid function to convert linear output into probability values. If probability > 0.5, the review is classified as positive; otherwise negative. The max_iter parameter is increased to ensure convergence.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (40000, 5000)
Testing size: (10000, 5000)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

What is Logistic Regression (Simple Explanation)

It calculates:

$z$

$w\ X + b$ z=wX+b

Then applies sigmoid function:

$\sigma ( z )$

$1\ 1 + e - z$ σ(z)= 1+e −z 1

Output ⟶ probability between 0 and 1.

STEP 6 — Model Evaluation

The model is evaluated using accuracy, precision, recall, and F1-score. Accuracy measures overall performance. Precision indicates how many predicted positive reviews were actually positive. Recall measures how many actual positive reviews were correctly identified. F1-score balances precision and recall. The confusion matrix visualizes correct and incorrect predictions.

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.8877
```

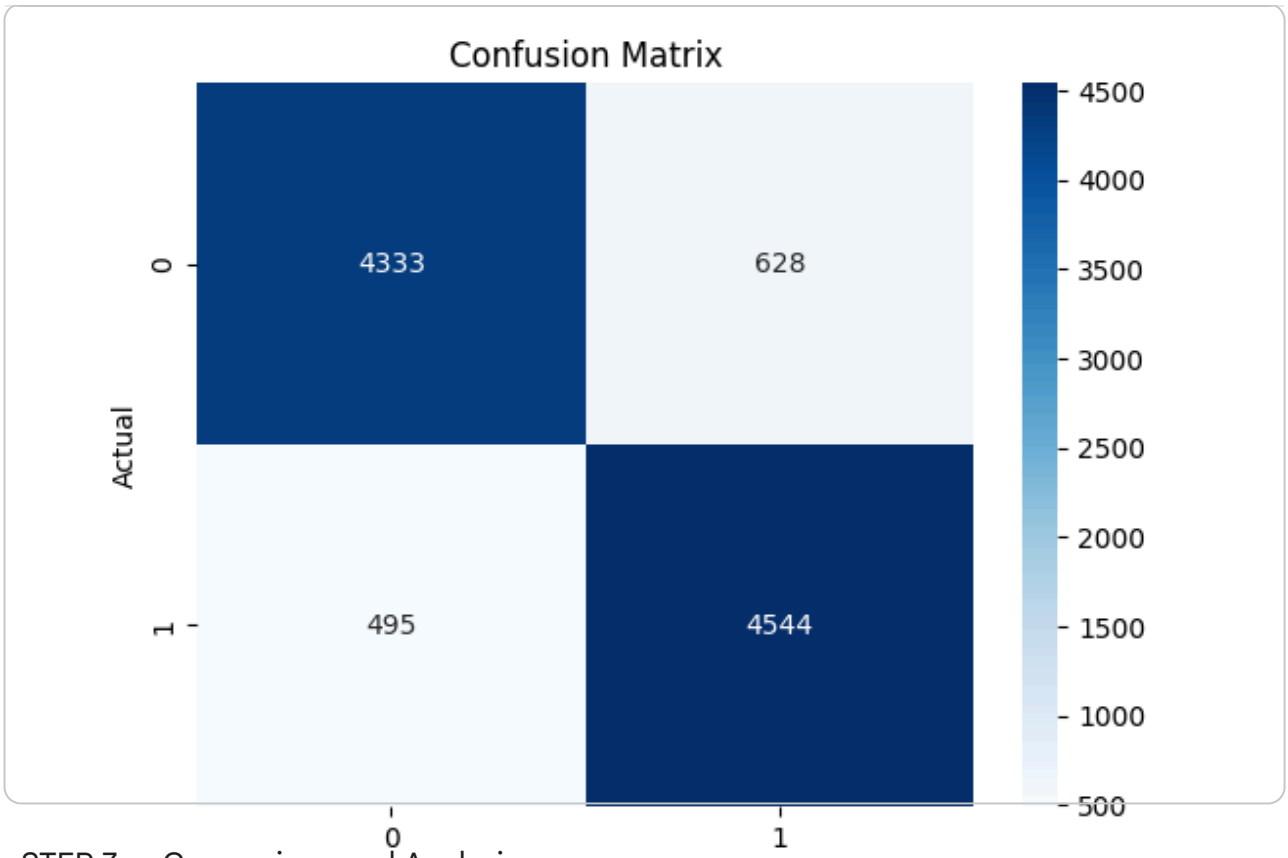
```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

    negative       0.90      0.87      0.89      4961
    positive       0.88      0.90      0.89      5039

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000
```

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

    negative       0.90      0.87      0.89      4961
    positive       0.88      0.90      0.89      5039

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




STEP 7 — Comparison and Analysis

Logistic Regression performed well for text classification using TF-IDF features. Compared to Naive Bayes, Logistic Regression generally achieves higher accuracy because it does not assume feature independence. Naive Bayes assumes that words are independent of each other, which is often unrealistic in natural language. Logistic Regression learns optimal feature weights and decision boundaries more effectively. TF-IDF features help Logistic Regression focus on sentiment-important words. Naive Bayes is computationally faster and simpler but may oversimplify word relationships. Logistic Regression usually performs better when sufficient training data is available. The confusion matrix shows balanced classification performance for positive and negative reviews. Overall, Logistic Regression is a strong discriminative model for sentiment analysis tasks.